Linear Models 2: Tree growth Lab

Dr. Luisa Barbanti and Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

Contents

1 Getting data

The dataset used here is about the growth rates of 557 trees. The response variable *growth.rate* represents growth rates measured via tree cores. Trees belong to four species. A few other variables expected to affect growth rates are also present in the dataset (e.g. "tree age"). Further information about the dataset can be found **[here]**.

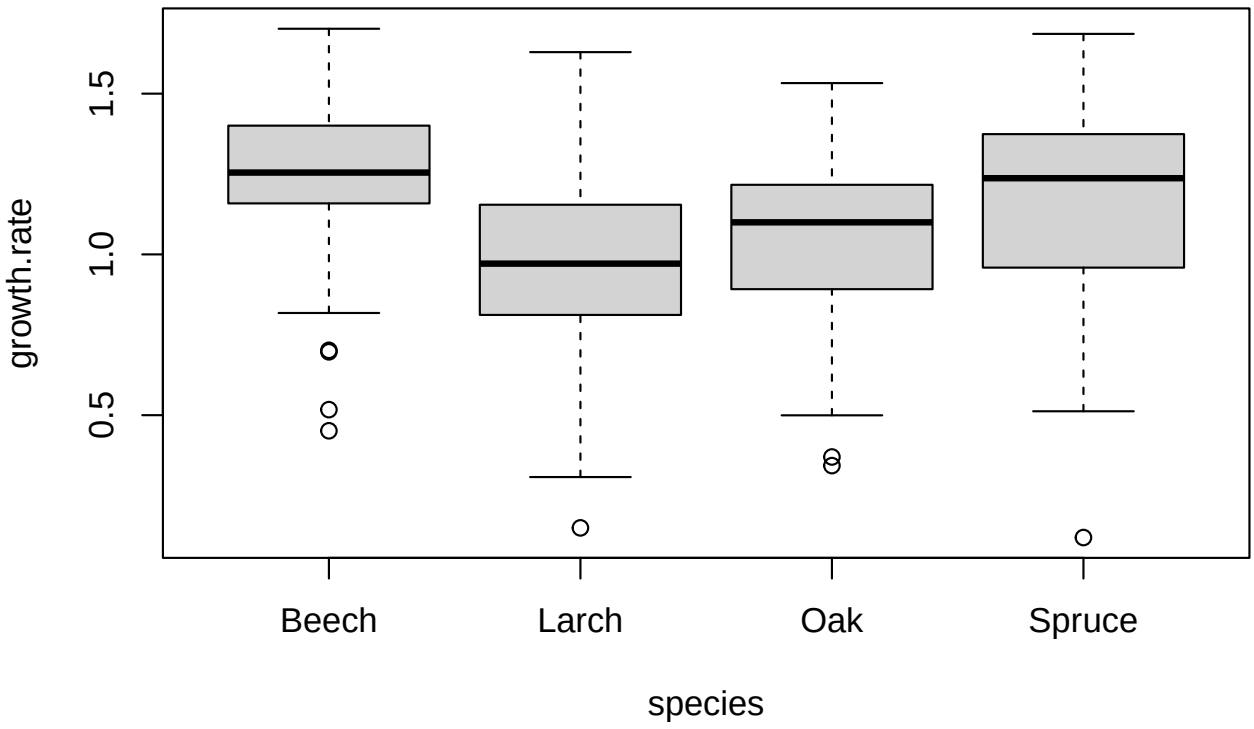
Let's load the data.

```
## (results are hidden)
##
d.trees <- read.csv2("TreesChamagne2017_Lab_modified.csv",
                     stringsAsFactors = TRUE)
##
str(d.trees)
head(d.trees)
```

2 Testing the effect of a categorical variable

Let's assume that we want to test whether different species grow at different rates. Let's visualise the data first. We use a "boxplot", a tool commonly used to visualise the effect of categorical variables.

```
boxplot(growth.rate ~ species, data = d.trees)
```



Growth seems to differ among species. Let's fit a linear model to these data and look at the estimated coefficients for the four species.

```
lm.trees.1 <- lm(growth.rate ~ species, data = d.trees)
coef(lm.trees.1)
```

```
  (Intercept)  speciesLarch    speciesOak speciesSpruce
       1.2528       -0.2996       -0.2009       -0.0867
```

You may remember that **R** uses "treatment contrasts". Therefore, the parameter named `(Intercept)` refers to the first species in alphabetical order (i.e. *Beech*), while the other coefficients represent the difference in intercepts to the reference level *Beech*.

Let's look at the p-values for each parameter present in the model.

```
summary(lm.trees.1)
```

```
Call:
lm(formula = growth.rate ~ species, data = d.trees)

Residuals:
    Min      1Q  Median      3Q     Max
-1.0466 -0.1486  0.0275  0.1778  0.6759

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)     1.2528     0.0216   58.05   <2e-16 ***
speciesLarch   -0.2996     0.0309   -9.71   <2e-16 ***
speciesOak     -0.2009     0.0305   -6.59    1e-10 ***
speciesSpruce  -0.0867     0.0309   -2.81   0.0051 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.257 on 553 degrees of freedom
Multiple R-squared:  0.163, Adjusted R-squared:  0.159
F-statistic:   36 on 3 and 553 DF,  p-value: <2e-16
```

The correct interpretation of the four p-values is:

- there is strong evidence that the mean growth rate for Beech (i.e. `(Intercept)`) is not equal to zero
- there is strong evidence that all the other three species mean growth rates differ from the reference level *Beech*

The comment that comes natural here is "why would we care about a difference between each species and *Beech*, which is an arbitrary reference?". Indeed, All the four p-values discussed here don't give us a direct answer to our actual question "do species differ in growth rates?".

To answer this latter question we must fit a model that consider species to have no effect at all. This is a very simple model that essentially computes the mean growth rate of all observations. So,it is a model that only contains an intercept. Let's fit this model.

What growth.rate ~ 1 means in R

1 means "include an intercept"
0 or -1 means "no intercept"

```
lm.trees.0 <- lm(growth.rate ~ 1, data = d.trees)
coef(lm.trees.0)
```

growth.rate ~ 1. means: Model growth.rate using only a single intercept (a constant mean), with no predictors. yi=β0+εi

```
(Intercept)
       1.11
```

In words "All observations are assumed to come from the same population with the same mean growth rate."

Let's now formally compare these two models with a F-test. To compare models in **R** we use the function `anova()`.

What does growth.rate ~ species do differently? That model is: yi=β0+β1I(Larch)+β2I(Oak)+β3I(Spruce)+εi

```
anova(lm.trees.0, lm.trees.1)
```

```
Analysis of Variance Table

Model 1: growth.rate ~ 1
Model 2: growth.rate ~ species
  Res.Df  RSS Df Sum of Sq  F Pr(>F)
1    556 43.7
2    553 36.6  3      7.14 36 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The output shows that there is strong evidence that the model with more parameters (i.e. one parameter for each species) better fits the data. Indeed, the "Residual Sums of Squares" (i.e. the unexplained variance, for short *RSS*) of the more complex model is clearly smaller. In other words, the more complex model explains way more of the variability of these data at a cost of only three additional parameters[1].

Note that, as a consequence of the clearly better fit, the F-value is extremely large and the p-value extremely small. Indeed, the F-test takes into consideration both aspects: The number of additional parameters and the drop in unexplained variance. Let's assume the drop in RSS was from 43.7 to 36.6, but to achieve that we should have added 100 parameters to our model, than the F-test would not have been significant.

So, in the case seen above we can claim that "there is very strong evidence that species differ in growth rates". Sometimes testing the overall effect of a categorical variable is our main interest and the analysis would stop here. However, we may also want to know more about the differences among the single levels. The results of this F-test just tells us that at least one species differs from the others (in terms of growth rate).

3 Contrasts

3.1 Oak vs. Spruce

Let's assume that we further investigate the species effect by testing the hypothesis that *Oak* differs from *Spruce* in terms of growth rates. To do that we are going to use the `glht()` function from the *{multcomp}* package[2].

To use the `glht()` function, we must specify three elements:

[1] The very simple model has only one parameter (i.e. the intercept). While `lm.trees.1` has four parameters (one for each species). Therefore, the second model has three more parameters. This information is printed in the `Df` column of the output above.

[2] There are several methods implemented in **R** to run contrasts. The *{multcomp}* package is currently the most reliable and flexible.

- the model
- which predictor is involved in the testing
- what levels of the latter predictor are to be compared

In the call below, we use the `lm.trees.1` model to test whether *Oak* and *Spruce* differ in terms of growth rates.

```
library(multcomp)
ph.test.1 <- glht(model = lm.trees.1,
                  linfct = mcp(species =
                                 c("Oak - Spruce = 0")))
summary(ph.test.1)
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: User-defined Contrasts


Fit: lm(formula = growth.rate ~ species, data = d.trees)

Linear Hypotheses:
                   Estimate Std. Error t value Pr(>|t|)
Oak - Spruce == 0  -0.1141     0.0308   -3.71  0.00023 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

The output is quite clear again. There is strong evidence that the growth rates differ between these two species. *Spruce* grows at a rate that is 0.114 larger than the rate of *Oak*. These results are in complete agreement with the graphical analysis carried out above (i.e. the boxplot).

See the *Appendix* for further examples of contrasts.

---

4 Testing several variables

4.1 Testing categorical variables

We tested the effect of *species* using the `anova()` function fed with two models. In general, we often want to test several variables present in a model.

Let assume the example that we have a model with two further predictors. *Density.tree.Class* is a predictor that measures the density of trees around the focal tree for which the growth rate was measured. This latter categorical predictor has three classes (i.e. "low", "medium" and "high").

SiteID is another categorical variable that accounts for the fact that the 557 trees were clustered into 45 groups (i.e. sites).

Let's add these variables to the model. To do that, we could refit the model via the `lm()` function or "update" the current model by using the `update()` function.

```
lm.trees.2 <- update(lm.trees.1,
                     . ~ . + Density.tree.Class + SiteID)
```

The ". ~ ." part in the call above means "refit the model with the same response variable" (the dot before tilde) and "with all the predictors that are already in the model" (the dot after tilde). Then the "+ Density.tree + SiteID" indicates which predictors we want to add to `lm.trees.2` model. Note that "-" can also be used to drop terms from a model.

Let's check that the model was refitted as we wished.

```
formula(lm.trees.2)
```

```
growth.rate ~ species + Density.tree.Class + SiteID
```

To test these two newly added categorical variables we could perform two F-tests via the `anova()` function. However, this is not needed as the `drop1()` function performs this automatically for each variable present in a given model.

```
drop1(lm.trees.2, test = "F")
```

```
Single term deletions

Model:
growth.rate ~ species + Density.tree.Class + SiteID
                   Df Sum of Sq  RSS   AIC F value Pr(>F)
<none>                          36.4 -1505
species             3      6.85 43.3 -1415   34.51 <2e-16 ***
Density.tree.Class  2      0.09 36.5 -1508    0.65   0.52
SiteID              1      0.08 36.5 -1506    1.24   0.27
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

You can convince yourself that the testing carried out with the `drop1()` function is fully equivalent to the use of the `anova()` function fed with two models by running all three corresponding tests. **Note that *drop1()* really drops one variable at the time**. This implies that, for example, the last line of the output above refers to a model with *SiteID* and a model without *SiteID*, but for both models compared on that line of output the other two variables (*species* and *Density.tree.Class*) are included in the model.

Question: *why is the F-value for the predictor* species *slightly different from the one obtained above when testing the* lm.trees.1 *model?*

The reason is that the effect of *species* is now estimated while taking into account the effect of the two other predictors in the current model (i.e. density and site).

Going back to the p-values obtained with the `drop1()` call above: The categorical predictor *Density.tree.Class* does not seem to have a relevant effect on the response variable (p-value is $\gg$ than 0.1)

Finally, the predictor *SiteID* does not seem to have a relevant effect either. However, note that the `Df` for this predictor is 1 instead of 44 as expected (i.e. number of levels of the categorical variable minus one).

Question: *Why is `Df` one here?*

The reason is that this variable was not correctly coded as being a categorical variables. Indeed, sites are numbered and therefore this predictor was considered by **R** to be a plain continuous variable.

```
head(unique(d.trees$SiteID))
```

```
[1]  1 12 15 18 23 24
```

To fix this problem we must declare this predictor to be a factor.

```
d.trees$SiteID.fac <- factor(d.trees$SiteID)
```

We can now refit the model and perform inference via the `drop1()` function.

```
lm.trees.3 <- update(lm.trees.2,
                     . ~ . + SiteID.fac - SiteID)
##
drop1(lm.trees.3, test = "F")
```

```
Single term deletions

Model:
growth.rate ~ species + Density.tree.Class + SiteID.fac
                  Df Sum of Sq  RSS   AIC F value  Pr(>F)
<none>                        29.4 -1539
species            3      3.97 33.3 -1474   22.87 6.7e-14 ***
Density.tree.Class  2      0.26 29.6 -1538    2.20    0.11
SiteID.fac         44      7.12 36.5 -1506    2.79 3.4e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The summary output of `drop1()` makes now sense (`Df` for site is 44). There seems to be quite some differences among sites.

The p-values for the "tree density class" predictor also changed and indicates that it might have an effect on the response variable.

4.2 Testing continuous and categorical variables

We have previously seen that to test continuous and dummy variables (i.e. categorical variable with two levels) we can use the `summary()` function (via the t-test). Note that `drop1()` can also be used with continuous predictors.

Let's add the *age* predictor.

```
lm.trees.4 <- update(lm.trees.3, . ~ . + age)
##
drop1(lm.trees.4, test = "F")
```

```
Single term deletions

Model:
growth.rate ~ species + Density.tree.Class + SiteID.fac + age
                  Df Sum of Sq  RSS   AIC F value  Pr(>F)
<none>                        29.3 -1538
```

```
species             3      4.02 33.3 -1473   23.10 5.0e-14 ***
Density.tree.Class  2      0.24 29.6 -1538    2.05    0.13
SiteID.fac         44      7.13 36.4 -1505    2.80 3.2e-08 ***
age                 1      0.05 29.4 -1539    0.84    0.36
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The output tells us that the *age* predictor does not seem to play an important role. To know whether this predictor has a positive or negative effect on the response variable, we can use the `coef()` function.

Note that the p-value for *age* obtained when using the *summary()* function is identical to the one obtained with *drop1()* (any differences here are due to different rounding).

```
capture.output(summary(lm.trees.4))[c(10,11,62)]
```

```
[1] "Coefficients:"
[2] "                         Estimate Std. Error t value Pr(>|t|)    "
[3] "age                       0.00149    0.00162    0.92   0.3585    "
```

So, to recapitulate the testing of continuous and categorical variables:

- continuous variables can be tested with the `summary()` function (i.e. via t-tests)
- continuous variables can equivalently be tested with the `drop1()` function (i.e. via F-tests)
- for a continuous variable the results of a t-test or a F-test are identical
- to test a categorical variable with two levels only (also called dummy variables) one can use the `summary()` function or the `drop1()` function (results are identical)
- to test a categorical variable with more than two levels the `drop1()` function must be used
- the `summary()` function used to test a categorical variable with more than two levels only tells us, whether the intercept differs from zero (most of the time not interesting) and whether there is a difference between the reference level and the other levels.

5 Appendix

5.1 Testing all predictors in a model

Sometimes, it may be of interest to test of any of the predictors have an influence on the response variable. To do that we can compare the full model and very simple model that only contains an intercept. This test, rarely used in practice, is often referred as "global F-test".

```
anova(lm.trees.0, lm.trees.4)
```

```
Analysis of Variance Table

Model 1: growth.rate ~ 1
Model 2: growth.rate ~ species + Density.tree.Class + SiteID.fac + age
  Res.Df  RSS Df Sum of Sq    F Pr(>F)
1    556 43.7
2    506 29.3 50      14.4 4.97 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Not surprisingly, there is very strong evidence that at least one predictors plays a relevant role.

Note that this F-test is actually reported by default at the very end of the the "summary output".

```
tail(capture.output(summary(lm.trees.4)))
```

```
[1] "Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1"
[2] ""
[3] "Residual standard error: 0.241 on 506 degrees of freedom"
[4] "Multiple R-squared:  0.33,\tAdjusted R-squared:  0.263 "
[5] "F-statistic: 4.97 on 50 and 506 DF,  p-value: <2e-16"
[6] ""
```

5.2 Sequential sums of squares (*)

The `anova()` function can also be fed with a single model. This would implement the sequential sums of squares Anova (also called "type I sums of squares"). The results obtained with this type of analysis depends on the ordering in which the predictors enter the formula[3]. So, writing "y ~ x1 + x2" would give different results to "y ~ x2 + x1". Consider the example below where we run the `anova()` function fed with a single model and the we redo the same thing after moving the *species* predictor to the last position.

```
formula(lm.trees.4)
```

```
growth.rate ~ species + Density.tree.Class + SiteID.fac + age
```

```
anova(lm.trees.4)
```

[3] Unless the design is orthogonal, which is virtually never the case in real settings.

```
Analysis of Variance Table

Response: growth.rate
                   Df Sum Sq Mean Sq F value  Pr(>F)
species             3   7.14   2.381   41.10 < 2e-16 ***
Density.tree.Class  2   0.09   0.047    0.81    0.44
SiteID.fac         44   7.12   0.162    2.79 3.4e-08 ***
age                 1   0.05   0.049    0.84    0.36
Residuals         506  29.31   0.058
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Let's now move the *species* predictor to the end of the formula call.

```
lm.trees.4.again <- lm(growth.rate ~ Density.tree.Class + SiteID.fac +
                         age + species,
                       data = d.trees)
anova(lm.trees.4.again)
```

```
Analysis of Variance Table

Response: growth.rate
                   Df Sum Sq Mean Sq F value  Pr(>F)
Density.tree.Class  2   0.42   0.212    3.66   0.026 *
SiteID.fac         44   9.96   0.226    3.91 4.1e-14 ***
age                 1   0.01   0.007    0.12   0.728
species             3   4.02   1.338   23.10 5.0e-14 ***
Residuals         506  29.31   0.058
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Comparing the two outputs we note that all p-values changed. For example, the p-value for the *Density.tree.Class* predictor went from 44% to 2.6%. This is a dramatic change and would have strong implications on the conclusions drawn here. **Therefore, this way to test predictors should be avoided.**

On the other hand, the results obtained with the `drop1()` function are unaffected from the ordering of the predictors (run the `drop1()` function on `lm.trees.4` and on `lm.trees.4.again` to convince yourself).

5.3 Specifying linear hypotheses via matrices of coefficients

Just below we reproduce the code and output used to compare *Oak* and *Spruce*.

```
library(multcomp)
ph.test.1 <- glht(model = lm.trees.1,
                  linfct = mcp(species =
                               c("Oak - Spruce = 0")))
summary(ph.test.1)
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: User-defined Contrasts
```

```
Fit: lm(formula = growth.rate ~ species, data = d.trees)

Linear Hypotheses:
                  Estimate Std. Error t value Pr(>|t|)
Oak - Spruce == 0  -0.1141     0.0308   -3.71  0.00023 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

This call uses the so-called "symbolic description". The other option is to use a matrix of coefficients. In this specific case there is just one hypothesis to test, so we create a Matrix of one row (ie. a vector).

```
levels(d.trees$species)
```

```
[1] "Beech"  "Larch"  "Oak"    "Spruce"
```

```
##
## vector of contrasts
v.Oak_vs_Spruce <- c(0, 0, -1, 1)
names(v.Oak_vs_Spruce) <-
  levels(d.trees$species)
##
v.Oak_vs_Spruce
```

```
 Beech  Larch    Oak Spruce
     0      0     -1      1
```

We then use this vector to test the hypothesis.

```
library(multcomp)
ph.test.Q_vs_P <- glht(model = lm.trees.1,
                  linfct = mcp(species = v.Oak_vs_Spruce))
summary(ph.test.Q_vs_P)
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: User-defined Contrasts


Fit: lm(formula = growth.rate ~ species, data = d.trees)

Linear Hypotheses:
       Estimate Std. Error t value Pr(>|t|)
1 == 0   0.1141     0.0308    3.71  0.00023 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

The matrix notation allows us to fit more complex contrasts (see next).

5.4 Testing broadleaved versus conifers (*)

To now test the difference between "broadleaved species" and "conifers" (i.e. *Beech* and *Oak* versus *Spruce* and *Larch*). Note that we are using a vector to specify this hypothesis.

```
levels(d.trees$species)
```

```
[1] "Beech"  "Larch"  "Oak"    "Spruce"
```

```
##
## vector of contrasts
v.conifers_vs_broadleaved <- c(1/2, -1/2, -1/2, 1/2)
names(v.conifers_vs_broadleaved) <-
  levels(d.trees$species)
##
v.conifers_vs_broadleaved
```

```
 Beech  Larch    Oak Spruce
   0.5   -0.5   -0.5    0.5
```

Let's feed the `glht()` function to test the contrasts of conifers versus broadleaved species.

```
ph.test.conifers_vs_broadleaved <- glht(
  model = lm.trees.1,
  linfct = mcp(species = v.conifers_vs_broadleaved))
##
summary(ph.test.conifers_vs_broadleaved)
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: User-defined Contrasts


Fit: lm(formula = growth.rate ~ species, data = d.trees)

Linear Hypotheses:
       Estimate Std. Error t value Pr(>|t|)
1 == 0   0.2068     0.0218    9.49   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

On average broadleaved species have higher growth rates (0.09 faster grow rates). There is strong evidence that this difference is significant.

5.5 Testing all pairwise comparisons

Let's assume that we may want to test all species in a pairwise manner. We have four species, we thus have 6 possible pairs of species. What if we had ten species? In that case, we would have had 45 pairs (i.e. $\frac{n \cdot (n-1)}{2}$).

When performing a large number of tests, we incur in the issue of **multiple testing**. Indeed, by running many tests, it may happen that some of them are significant just do to chance[4].

Fortunately, there are ways to control for the number of tests performed and avoid the multiple testing issue. In the specific case of testing all possible pairs of species, we can use the "Tukey Honest Significant Difference Test" method.

```
ph.test.THSD <- glht(lm.trees.1,
                     linfct = mcp(species = "Tukey"))
summary(ph.test.THSD)
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: Tukey Contrasts


Fit: lm(formula = growth.rate ~ species, data = d.trees)

Linear Hypotheses:
                     Estimate Std. Error t value Pr(>|t|)
Larch - Beech == 0   -0.2996     0.0309   -9.71   <0.001 ***
Oak - Beech == 0     -0.2009     0.0305   -6.59   <0.001 ***
Spruce - Beech == 0  -0.0867     0.0309   -2.81   0.0259 *
Oak - Larch == 0      0.0987     0.0308    3.20   0.0077 **
Spruce - Larch == 0   0.2128     0.0312    6.82   <0.001 ***
Spruce - Oak == 0     0.1141     0.0308    3.71   0.0014 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

Each row in this output show a pairwise comparison between two species. Not unexpectedly, we see that there almost all tests are highly significant. Note that p-values reported here are corrected for the number of tests performed. Indeed, the p-value for the "Oak - Spruce" test here is larger than the one obtained in the test above due to the correction.

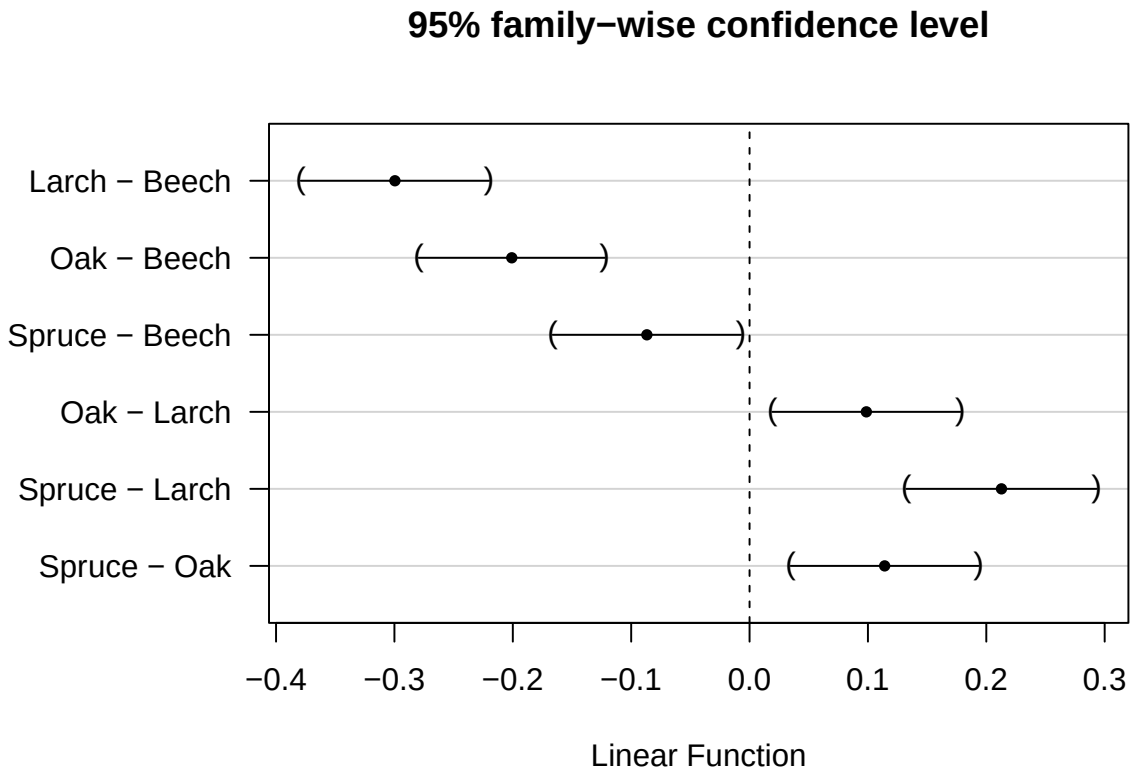
All these tests can also be visualised in a graphical manner. Note that the margins of the plotting area are adjusted to make all contrast names fit into it.

```
## 1) change margins default
par("mar")
```

```
[1] 5.1 4.1 4.1 2.1
```

```
par(mar = c(5.1,7.5,4.1,2.1))
##
## 2) plot contrasts
plot(ph.test.THSD)
```

[4]By running 100 tests under the null hypothesis of no effect, we expect five of them to be significant at the 5% level. To convince yourself, you can simulate data accordingly.



The above graph shows all the estimated pairwise differences (i.e. the dots) and the associated confidence intervals.

5.6 Testing several contrasts (**)

Sometimes we may want to test several contrasts. For example we may want to test the following hypotheses:

- *Beech* grows faster than *Larch*
- *Spruce* grows faster than *Oak*
- *Beech* grows faster than the average of *Oak* & *Spruce*
- the growth difference "Beech - Larch" is larger than the growth difference "Oak - Spruce".

```
M.ph.test.mat.1 <- rbind(
  "Beech vs Larch" = c(1, -1, 0, 0),
  "Spruce vs Oak" = c(0, 0, 1,-1),
  "Beech vs Oak & Spruce" = c(1, 0, -1/2, -1/2),
  "F - L vs Q - P" = c(1, -1, 1, -1))
colnames(M.ph.test.mat.1) <- levels(d.trees$species)
##
M.ph.test.mat.1
```

```
                      Beech Larch  Oak Spruce
Beech vs Larch            1    -1  0.0    0.0
Spruce vs Oak             0     0  1.0   -1.0
Beech vs Oak & Spruce     1     0 -0.5   -0.5
F - L vs Q - P            1    -1  1.0   -1.0
```

Let's perform the testing.

```
ph.test.mat.1 <- glht(
  model = lm.trees.1,
  linfct = mcp(species = M.ph.test.mat.1))
##
summary(ph.test.mat.1)
```

```
     Simultaneous Tests for General Linear Hypotheses

Multiple Comparisons of Means: User-defined Contrasts


Fit: lm(formula = growth.rate ~ species, data = d.trees)

Linear Hypotheses:
                           Estimate Std. Error t value Pr(>|t|)
Beech vs Larch == 0          0.2996     0.0309    9.71   <0.001 ***
Spruce vs Oak == 0          -0.1141     0.0308   -3.71   <0.001 ***
Beech vs Oak & Spruce == 0   0.1438     0.0265    5.42   <0.001 ***
F - L vs Q - P == 0          0.1854     0.0436    4.25   <0.001 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
(Adjusted p values reported -- single-step method)
```

Note that running each contrast alone would lead to the same estimates, but to different p-values. Only by running all contrasts together in one call makes sure that p-values are properly corrected for the total number of tests (see very last line of the output for the correction method used).

5.7 Conifers vs broadleaved, why not a t-test? (*)

Instead of fitting a linear model and then perform contrasts, we may want to run a simple t-test to compare the growth of these two groups of species. There are several reasons why running a t-test is inferior to fit a linear model and than use contrasts.

1. *Reduced power:* by fitting a model we can control for the effect of many other predictors. In this example, we may want to estimated the broadleaved-conifers difference after controlling for the noise brought in by e.g. "tree age" and "tree density". In other words, by fitting a model and then using contrasts the difference can be estimated with an higher precision than in a t-test. This also implies that we have an higher likelihood to detect a significant effect when it is present.

2. *Non-independence of the observations:* very often some design variables are present and must be considered. For example, here we must include the "site" predictor into the model. If we where to run a t-test, where we assume independence of the observations, the observations that come the same site would erroneously be considered to be independent.

3. *Multiple testing:* if several tests are carried out, p-values correction should be performed. This is feasible, but in practice more difficult if we were to run several t-tests.

4. *Testing non-sensical hypothesis:* Not all t-test we may want to perform may be sensible. Assume we are interested in the effect of "density low" vs "density high". If the predictor density is involved in any

significant interaction, then we should take this into account. For example, here the effect of density seemed to depend on species. Therefore, we should rather perform the test "density low" vs "density high" for each species separately. If we were to run t-tests disconnected from the model, we may end up testing hypotheses that should not be tested. Here, we may test "density low" vs "density high" by forgetting that this may not be a sensible choice as density effect varies from species to species.

5.8 Conifers vs broadleaved, why not adding a dummy variable? (***)

Following on the argumentation above, we may say that to test the effect of conifers vs broad-leaved, we simply add a dummy variable that signals whether the species is a conifer or a broad-leaved species to the original linear model (that contains species and all other predictors).

```
d.trees$conifers.YES <- d.trees$species %in% c("Larch", "Spruce")
##
table(d.trees$species,
      d.trees$conifers.YES)
```

```
         FALSE TRUE
  Beech    142    0
  Larch      0  136
  Oak      143    0
  Spruce     0  136
```

Let's fit a model with the new predictor.

```
lm.trees.conifers <- update(lm.trees.1, . ~ . + conifers.YES)
##
summary(lm.trees.conifers)
```

```

Call:
lm(formula = growth.rate ~ species + conifers.YES, data = d.trees)

Residuals:
    Min      1Q  Median      3Q     Max
-1.0466 -0.1486  0.0275  0.1778  0.6759

Coefficients: (1 not defined because of singularities)
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        1.2528     0.0216   58.05   <2e-16 ***
speciesLarch      -0.2996     0.0309   -9.71   <2e-16 ***
speciesOak        -0.2009     0.0305   -6.59    1e-10 ***
speciesSpruce     -0.0867     0.0309   -2.81   0.0051 **
conifers.YESTRUE      NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.257 on 553 degrees of freedom
Multiple R-squared:  0.163, Adjusted R-squared:  0.159
F-statistic:   36 on 3 and 553 DF,  p-value: <2e-16
```

Unfortunately, this model is said to be "rank-deficient" and therefore not all parameters can be estimated. Rank-deficiency implies that the design matrix of the model is not of full rank. This means that one column of the design matrix is a linear combination of the others.

The concept of rank-deficiency is not straightforward, so to give you an intuition consider the following argument.

In the *lm.trees.conifers* model here we tried to estimate five parameters (an intercept, three additional parameters for *species* AND another one for *conifers.YES*) from four groups of observations only (indeed, there are four species). In other words, we tried to estimate more parameters than available groups, which leads to the problem.

In practical terms, this implies that having knowledge of the tree species allows one to directly determine whether it is a conifer or a broad-leaved tree, without having to store this information inside another variable.

Let's illustrate this with an example. Consider a data frame with two variables, *species* and *living_being*. The species variable encompasses various types of animals and plants, while the *living_being* variable indicates whether the specific species is an animal or a plant.

For instance, if the *living_being* variable takes the value "dog" for an observation, it unequivocally indicates that the value of the second variable is "animal". Consequently, the second variable provides redundant information, and removing it does not alter the information available.

To illustrate this concept with our example, we create a small data set with a unique observation for each tree type, fit the same model as before, and display the model matrix.

5.9 Principle of marginality (**)

Note that the *drop1()* function respects the principal of marginality. To explain what this means, we add an interaction to our model. Let's assume that we would expect different species to be affected in different ways by tree age. To properly model that we must introduce the interaction term between these two predictors.

```
lm.trees.5 <- update(lm.trees.4,
                     . ~ . + age:species)
##
drop1(lm.trees.5, test = "F")
```

```
Single term deletions

Model:
growth.rate ~ species + Density.tree.Class + SiteID.fac + age +
    species:age
                  Df Sum of Sq  RSS   AIC F value  Pr(>F)
<none>                         28.0 -1558
Density.tree.Class  2      0.22 28.2 -1557    2.01    0.14
SiteID.fac         44      7.79 35.8 -1509    3.18 3.4e-10 ***
species:age         3      1.31 29.3 -1538    7.85 3.9e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

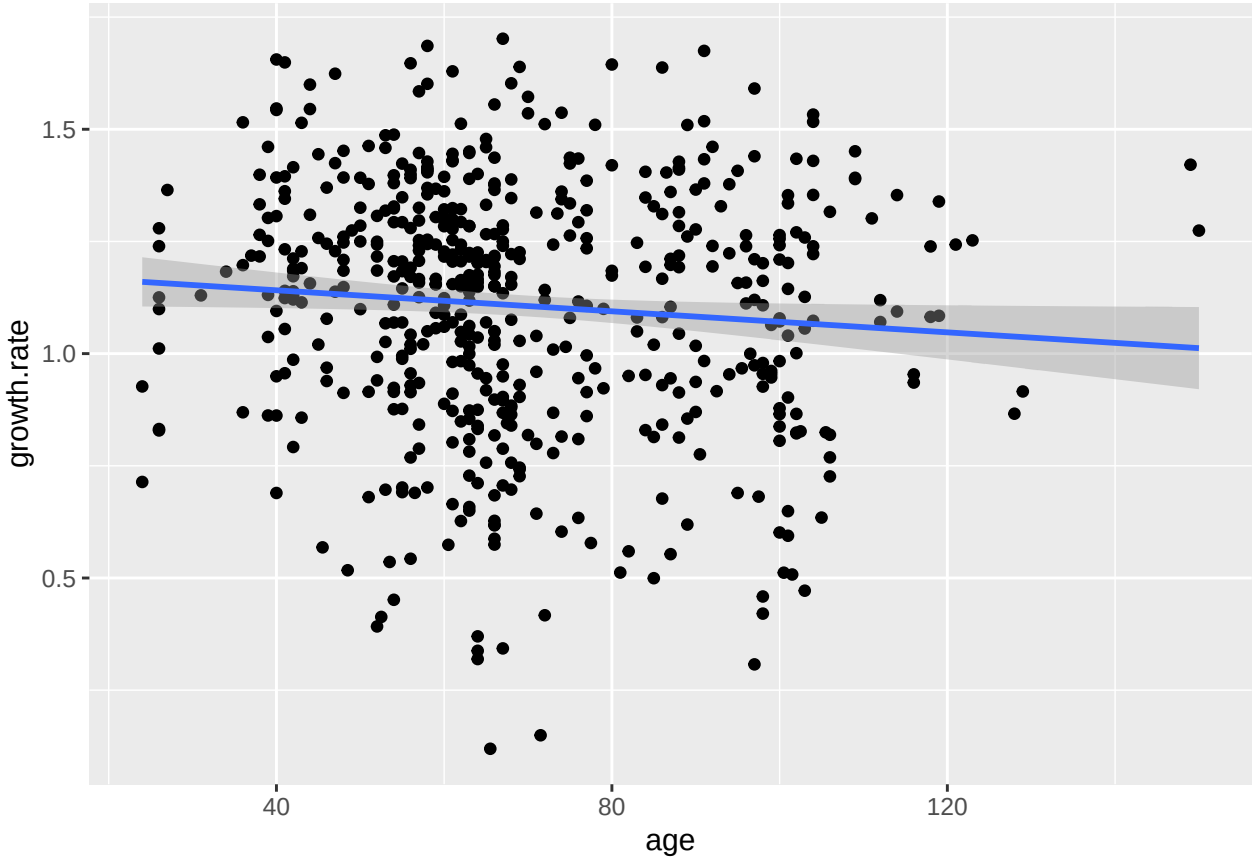
There is strong evidence that the age interacts with species. In other words, the effect of age is different among species.

Question: *Why was then the effect of* age *non-significant in the previous model?*

The reason is that the effect of *age* may be absent or very weak when averaged over all species. However, it may be that two species have a strong positive effect and two other species have a strong negative effect. Averaged, this gives no overall effect.

Let's visualise the effect of *age* for all species together first. Note that we are using the add-on package *{ggplot2}*. The use of the *ggplot()* function will be discussed later in this course.

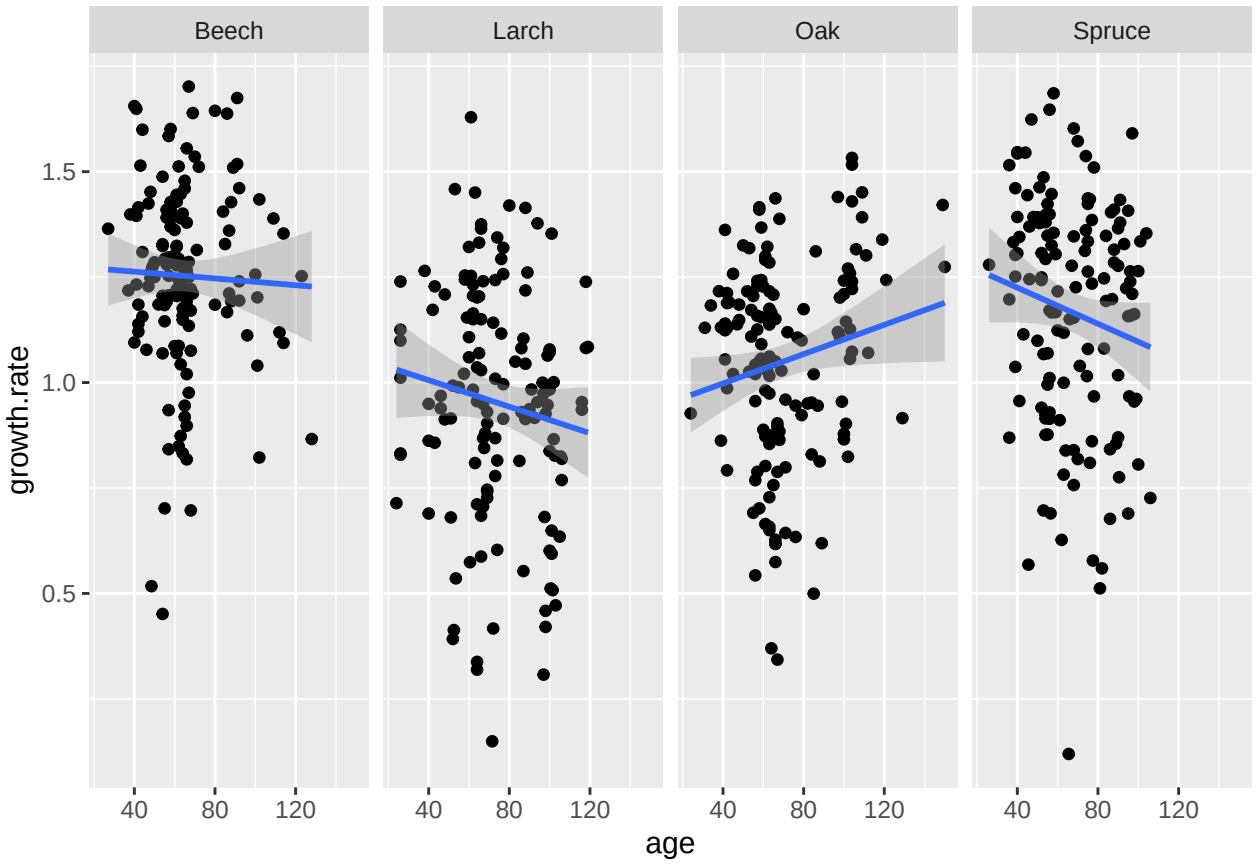
```
library(ggplot2)
ggplot(data = d.trees,
       mapping = aes(y = growth.rate,
                     x = age)) +
  geom_point() + ## adds observations
  geom_smooth(method = "lm") ## adds the regression line with CI
```



When species are ignored, the effect of *age* seems to be weak. Let's visualise each species in a different graph.

```
ggplot(data = d.trees,
       mapping = aes(y = growth.rate,
                     x = age)) +
  geom_point() +
  geom_smooth(method = "lm") +
  facet_grid(. ~ species)
```

```
`geom_smooth()` using formula = 'y ~ x'
```



These four plots show clearly that the effect of *age* is not constant across species. For example, "Larch" trees seem decrease their growth rates over age, while "Oak" show the opposite pattern.

So,the right conclusion to draw here is that "there is strong evidence that *age* has an effect on growth rate and this effect differs among species."

This conclusion implies that both predictors involved in the interaction are relevant. In other words, as soon as an interaction term is significant, both predictors are to be considered relevant. and no further testing is required to assess whether they play a role.

This is why the *drop1()* function is not testing for main effects that are involved in an interaction. Indeed, in the output above neither *age* nor *species* are tested.

This is the principle of marginality. Main effects must be tested only if they are not involved in interactions or higher-degree terms (such as polynomials; see week three). More in general, marginality implies the testing of higher-degree terms first.

5.10 Comparing F-tests and t-tests for categorical variables

In section 2 we fitted a very simple model that only contained the "species" predictor (i.e. *species*). We first looked at the summary output (t-tests) and then at the output of the *drop1()* function (F-tests). We reproduce these results here.

```
summary(lm.trees.1)
```

```
Call:
lm(formula = growth.rate ~ species, data = d.trees)

Residuals:
    Min      1Q  Median      3Q     Max
-1.0466 -0.1486  0.0275  0.1778  0.6759

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)     1.2528     0.0216   58.05   <2e-16 ***
speciesLarch   -0.2996     0.0309   -9.71   <2e-16 ***
speciesOak     -0.2009     0.0305   -6.59    1e-10 ***
speciesSpruce  -0.0867     0.0309   -2.81   0.0051 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.257 on 553 degrees of freedom
Multiple R-squared:  0.163, Adjusted R-squared:  0.159
F-statistic:   36 on 3 and 553 DF,  p-value: <2e-16
```

```
##
drop1(lm.trees.1, test = "F")
```

```
Single term deletions

Model:
growth.rate ~ species
        Df Sum of Sq  RSS   AIC F value Pr(>F)
<none>               36.6 -1509
species  3      7.14 43.7 -1415      36 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

By looking at these results one may ask him/herself whether we really need to run but type of tests as we already saw in the summary output that there are differences among species.

The answer to this question is yes, we need to run both tests simply because there are testing different things. The t-tests in the summary output test whether the species "Larch", "Spruce" and "Oak" differ from the reference level "Beech". while the F-test focuses on the more general question "are there any differences among species".

To better appreciate the difference between the two tests we can change the reference level in our model and run both tests again. Let's set "Oak" as the reference level and refit the model.

```
d.trees$species.relevelled <- relevel(d.trees$species,
                                      ref = "Oak")
##
lm.trees.1.relevelled <- update(lm.trees.1,
                                . ~ . -species + species.relevelled)
```

Let's now look at the p-values of the t-test and F-test.

```r
summary(lm.trees.1.relevelled)
```

```
Call:
lm(formula = growth.rate ~ species.relevelled, data = d.trees)

Residuals:
    Min      1Q  Median      3Q     Max
-1.0466 -0.1486  0.0275  0.1778  0.6759

Coefficients:
                         Estimate Std. Error t value Pr(>|t|)
(Intercept)                1.0519     0.0215   48.91  < 2e-16 ***
species.relevelledBeech    0.2009     0.0305    6.59    1e-10 ***
species.relevelledLarch   -0.0987     0.0308   -3.20  0.00144 **
species.relevelledSpruce   0.1141     0.0308    3.71  0.00023 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.257 on 553 degrees of freedom
Multiple R-squared:  0.163, Adjusted R-squared:  0.159
F-statistic:   36 on 3 and 553 DF,  p-value: <2e-16
```

```r
##
drop1(lm.trees.1.relevelled, test = "F")
```

```
Single term deletions

Model:
growth.rate ~ species.relevelled
                  Df Sum of Sq  RSS   AIC F value Pr(>F)
<none>                         36.6 -1509
species.relevelled  3      7.14 43.7 -1415      36 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interestingly, the p-values from the summary output are different as the reference level changed. The only p-values that is identical is the one comparing "Oak" and "Beech" as this test was already present in the summary of the *lm.trees.1* model.

On the other hand, the p-value of the F-test run via the *drop1()* function is identical for both models (*lm.trees.1* and *lm.trees.1.relevelled*). Indeed, the question addressed by the F-test "are there any differences among species?" is unaffected by the reference level.

6 Session Information

```r
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
```

```
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.6.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK versi

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Zurich
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] ggplot2_3.5.2   multcomp_1.4-28 TH.data_1.1-3   MASS_7.3-65
[5] survival_3.8-3  mvtnorm_1.3-3   knitr_1.50

loaded via a namespace (and not attached):
 [1] vctrs_0.6.5        nlme_3.1-168       cli_3.6.5          rlang_1.1.6
 [5] xfun_0.52          generics_0.1.4     labeling_0.4.3     zoo_1.8-14
 [9] glue_1.8.0         htmltools_0.5.8.1  tinytex_0.57       scales_1.4.0
[13] rmarkdown_2.29     grid_4.5.1         tibble_3.3.0       evaluate_1.0.4
[17] fastmap_1.2.0      yaml_2.3.10        lifecycle_1.0.4    compiler_4.5.1
[21] dplyr_1.1.4        codetools_0.2-20   RColorBrewer_1.1-3 sandwich_3.1-1
[25] pkgconfig_2.0.3    mgcv_1.9-3         rstudioapi_0.17.1  farver_2.1.2
[29] lattice_0.22-7     digest_0.6.37      R6_2.6.1           tidyselect_1.2.1
[33] pillar_1.11.0      splines_4.5.1      magrittr_2.0.3     Matrix_1.7-3
[37] withr_3.0.2        tools_4.5.1        gtable_0.3.6
```